

Lista 1

Relatório da Implementação dos Algoritmos de Busca no 8-puzzle (15-puzzle foi adicionado ao relatório mas não adicionado o código da implementação)

Disciplina: Inteligência Artificial
Aluno: Huina G. Pereira – Matrícula: 00330523
Data: 22/09/2025

Tabela de Resultados Médios – 8-puzzle

Algoritmo	Estados Testados	Média Nós Expandidos	Média Comprimento da Solução	Média Tempo (s)	Média h	h Inicial (média)
GBFS	100	392.4200	140.5200	0.0011	6.9068	13.8800
BFS	100	80962.5900	22.2800	0.1919	0	13.8800
IDFS	100	2578290.5600	22.1600	0.0659	0	13.8800
A*	100	895.0000	22.1600	0.0028	10.0338	13.8800
IDA*	100	2373.0300	22.1600	0.0052	13.8800	13.8800

Neste trabalho foi implementado diferentes algoritmos de busca (BFS-Graph, Iterative Deepening, A*, IDA* e Greedy Best-First Search) aplicados ao problema do 8-puzzle. Avaliando desempenho em termos de número de nós expandidos, comprimento da solução, tempo de execução e comportamento da heurística.

Todas as implementações foram realizadas em **C++**, compiladas via **Makefile** e obedecendo ao padrão de entrada e saída definido no enunciado.

A heurística utilizada foi a **distância Manhattan**, com registro do valor inicial e cálculo da média durante a execução. Cada algoritmo foi estruturado com as **estruturas de dados apropriadas** (filas, pilhas, priority_queue, unordered_set) e respeitou a ordem de geração de sucessores (cima, esquerda, direita, baixo) e os critérios de desempate especificados.

Os resultados foram apresentados em **tabela de médias** por algoritmo e detalhados para cada estado inicial, incluindo número de nós expandidos, comprimento da solução, tempo de execução, valor médio da heurística e valor da heurística no estado inicial. A execução foi avaliada em termos de **tempo e memória**, garantindo eficiência: menos de 10 minutos para o 8-puzzle.

Este relatório descreve as **estratégias de implementação, desafios enfrentados** e apresenta os resultados obtidos, atendendo aos requisitos do enunciado e servindo como base para análise

comparativa de desempenho dos algoritmos.

-Configuração da Máquina de Testes:

- Processador: Intel(R) Core(TM) i3-3240 CPU @ 3.40GHz
- Memória RAM: 3,96 GB
- Sistema Operacional: Microsoft Windows 10 Pro 10.0.19045
- Compilador: gcc 14.2.0 / g++ 14.2.0 (MinGW-W64)

Greedy Best-First Search (GBFS)

Estruturas de dados: priority_queue para selecionar o nó mais promissor e unordered_set para evitar ciclos.

Heurística: Distância Manhattan. Valor inicial (h0) registrado e média calculada ao longo da execução.

Ordem de geração de sucessores: cima, esquerda, direita, baixo.

Critérios de desempate: menor h → maior g → LIFO.

Reconstrução do caminho: cada nó mantém ponteiro para o pai; apenas o comprimento da solução é registrado.

Desafios: Evitar ciclos: Foi necessário implementar controle de estados visitados para não expandir repetidamente os mesmos estados.

Gerenciar memória dinâmica: Cada nó alocado dinamicamente exige cuidado para evitar vazamentos de memória, especialmente com grandes conjuntos de teste.

Implementar critérios de desempate consistentes.

• Resultados:

```
$ ./main -gbfs 0 6 1 7 4 2 3 8 5, 5 0 2 6 4 8 1 7 3, 2 4 7 0 3 6 8 1 5
```

16,16,0.0001563000,5.50000,10

96,27,0.0003133000,8.05455,11

90,57,0.0002813000,6.80982,15

Tempo total de processamento: 0.00 segundos

Memória aproximada usada: 0.02 MB

Busca em Largura com controle de estados visitados (BFS-Graph)

Estruturas de dados utilizadas

- **deque:** fila open com comportamento FIFO (First-In-First-Out) para expansão de nós.
- **unordered_set:** conjunto closed para armazenar estados já visitados e evitar ciclos.

Tratamento de estados visitados

- Cada estado é armazenado no unordered_set para evitar revisitação.
- É feita verificação do nó pai imediato para prevenir voltar ao estado anterior.

Heurística

- **Distância Manhattan** calculada apenas para registro.

Reconstrução do caminho da solução

- Cada nó (NoBusca) mantém ponteiro para o pai e custo do caminho.
- Permite reconstruir a solução retrocedendo pelos pais ao alcançar o estado objetivo.

Estratégia

- BFS garante solução ótima em número de movimentos.
- Controle de estados visitados minimiza expansão desnecessária de nós.

Desafios e inconsistências observadas

- Observações sobre variações nos resultados:
- Pequenas variações nos valores de nós expandidos e comprimento da solução podem ocorrer entre execuções.
- Possíveis causas incluem:
 - Ordem indefinida de iteração do `unordered_set`, que pode alterar a ordem de expansão de nós no mesmo nível.
 - Diferenças no gerenciamento de memória dos nós, especialmente na alocação e liberação de ponteiros.
- Apesar dessas variações, **a maioria das execuções produz resultados corretos**, com pequenas divergências em casos específicos porém compromete a consistência da implementação.

• Resultados:

```
./main -bfs 0 6 1 7 4 2 3 8 5, 5 0 2 6 4 8 1 7 3, 2 4 7 0 3 6 8 1 5
```

```
5209,16,0.0102035000,0,10
```

```
42704,21,0.1274326000,0,11
```

```
86288,23,0.2112348000,0,15
```

```
Resultados salvos em resultados_bfs.csv
```

```
Tempo total de processamento: 0.35 segundos
```

```
Memória aproximada usada: 9.73 MB
```

Iterative Deepening Depth-First Search (IDFS)

Estruturas de dados utilizadas

- `array<int,9>`: permite acesso direto a qualquer posição com **índice constante** ($O(1)$), essencial para operações repetidas de geração de sucessores e cálculo de heurísticas. Foi usado porque o problema tem tamanho fixo, isso melhora desempenho e simplifica a manipulação de estados na recursão do IDFS
- `vector<Estado>`: armazenagem do caminho encontrado.

- Recursão da DFS limitada para expansão de nós.

Funcionamento

- Executa **DFS limitada** com limite de profundidade crescente.
- A cada iteração, aumenta o limite em +1, garantindo que **nós rasos sejam explorados primeiro**, similar ao BFS.
- Evita ciclos imediatos impedindo que o espaço vazio volte à posição do pai.
- Verifica **solucionabilidade** via contagem de inversões antes de expandir o estado.

Heurística

- **Distância Manhattan** calculada apenas para registro; não influencia a busca.

Reconstrução do caminho da solução

- Os estados expandidos são empilhados recursivamente e, ao encontrar o objetivo, o caminho é invertido para formar start → goal.

Estratégias

- Combina **baixa memória do DFS** com **completude do BFS**.
- Evita armazenar todos os nós em memória, ao contrário do BFS.
- `nos_expandidos` contabiliza apenas os nós efetivamente expandidos em cada iteração.

Desafios da implementação

1. **Recursão profunda:** o limite de profundidade deve ser suficiente para alcançar soluções sem estourar a pilha.
2. **Evitar ciclos:** é necessário controlar o movimento reverso do zero para não gerar estados redundantes.
3. **Consistência na contagem de nós:** somar os nós expandidos de cada iteração requer cuidado para não duplicar a contagem.
4. **Parsing de entradas múltiplas:** espaços ou vírgulas extras podem criar estados inválidos ou incompletos.
5. **Estimação de memória:** feita a partir do número de nós expandidos, mas não inclui overhead da pilha de chamadas recursivas.

• Resultados:

```
$ ./main -idfs 0 6 1 7 4 2 3 8 5, 5 0 2 6 4 8 1 7 3, 2 4 7 0 3 6 8 1 5
```

```
20305,16,0.000601600,0,10
```

```
336804,21,0.008130300,0,11
```

```
1183127,23,0.028913600,0,15
```

Resultados salvos em resultados.csv

Tempo total de processamento: 0.04 segundos

Memória aproximada usada: 111.64 MB

A*

O algoritmo **A*** foi implementado utilizando a **heurística de distância Manhattan**. A prioridade na exploração dos nós segue a ordem **menor f** → **menor h** → **LIFO (último inserido primeiro)**, conforme especificado no enunciado. A geração de sucessores respeita a convenção **cima, esquerda, direita, baixo**, evitando recriar o estado pai do nó atual.

Para a gestão dos nós, foi utilizada uma **priority_queue** para o conjunto aberto e um **unordered_set** para os estados já visitados. A média da heurística foi calculada a partir de todos os nós gerados, permitindo acompanhar o comportamento da função heurística ao longo da execução.

Desafios calcular o valor médio da função heurística de forma consistente com o esperado pelo enunciado foi desafiador. Tivemos que acompanhar cuidadosamente cada chamada da heurística e acumular os valores, além de testar diferentes formas de contabilizar os nós gerados por causa de pequenas variações causadas pela ordem de desempate na **priority_queue**. Apesar dos esforços, **não conseguimos chegar exatamente ao valor de referência**, sendo a média obtida uma aproximação. Essa análise, no entanto, permitiu compreender melhor como a função heurística influencia a prioridade dos nós e o desempenho do algoritmo.

• Resultados:

```
$ ./main -astar 0 6 1 7 4 2 3 8 5, 5 0 2 6 4 8 1 7 3, 2 4 7 0 3 6 8 1 5
```

```
91,16,0.0003660000,6.73292,10
```

```
612,21,0.0021221000,9.83004,11
```

```
511,23,0.0016618000,9.98210,15
```

```
Resultados salvos em resultadosAstar.csv
```

```
Tempo total de processamento: 0.01 segundos
```

```
Memória aproximada usada: 0.12 MB
```

IDA*

Estruturas de dados utilizadas

- `vector<int>` para representar o estado do tabuleiro.
- `Node struct` para cada nó, contendo estado, custo (g), heurística (h), soma (f) e ponteiro para o nó pai.
- Contadores globais para monitorar chamadas da heurística e estatísticas de desempenho.

Tratamento de estados visitados

- Evita ciclos imediatos comparando o estado do sucessor com o nó pai.
- Não há armazenamento global de estados visitados; a gestão é implícita na recursão e nos ponteiros para pais.

Heurística

- **Distância de Manhattan**, ignorando a peça vazia.
- Chamadas contabilizadas para calcular média global e valor no estado inicial.

Estratégia

- Algoritmo **IDA***: busca em profundidade limitada com aumento iterativo do limite $f(n) = g(n) + h(n)$.
- Expansão de nós recursiva até que $f(n)$ ultrapasse o limite.
- Próximo limite definido pelo menor $f(n)$ excedente encontrado.
- Caminho rastreado via ponteiros para o pai, liberando memória dos nós descartados com delete.
- Solução ótima garantida pelo controle de profundidade (g) e heurística consistente.

Desafios e inconsistências observadas

- Contagem precisa de **nós expandidos** em cada instância.
- Evitar **ciclos** sem armazenar todos os estados visitados.
- Gerenciamento de **memória**, liberando nós descartados para não interferir em execuções subsequentes.
- Dependência da **ordem de geração de sucessores** para reprodutibilidade.
- Apesar dos esforços, **não conseguimos chegar exatamente ao valor de referência**, sendo a média obtida uma aproximação.

• Resultados:

\$./main -idastar 0 6 1 7 4 2 3 8 5, 5 0 2 6 4 8 1 7 3, 2 4 7 0 3 6 8 1 5

165,16,0.0007757000,7.11684,10
 439,21,0.0015371000,10.71238,11
 724,23,0.0026948000,11.71120,15
 Finalizado: 3 instâncias resolvidas.
 Saída salva em: resultadosldastar.csv
 Tempo total: 0.02 segundos
 Memória aproximada usada: 0.14 MB

A* com 15-puzzle

A implementação do 15-Puzzle com A* foi desafiador devido ao espaço de estados enorme (16! combinações), ao overhead de shared_ptr e alocação dinâmica, à necessidade de um closed set eficiente e à geração de sucessores sem cópias desnecessárias, armazenar estados visitados de forma rápida e eficiente é crucial para evitar revisitação e não travar a memória. Computadores com RAM limitada sofrem lentidão ou swap, tornando essencial otimizar armazenamento e hashing. Gerenciamento do conjunto fechado (closed set)

* implementação chegou a ser iniciada mas não finalizada.

```
Administrador@DESKTOP-3EFMKUS MINGW64 ~/Desktop/astar15_puzzle
$ ./main -astar 7 11 8 3 14 0 6 15 1 4 13 9 5 12 2 10, 12 9 0 6 8 3 5 14 2 4 11 7 10 1 15 13, 13 0 9 12
11 6 3 5 15 8 1 10 4 14 2 7
154092,46,0.5906461000,23.00901,36
2731989,50,25.5132340000,24.64770,32
744209,53,2.6855253000,28.06488,39
Resultados salvos em resultados.csv
Tempo total de processamento: 28.83 segundos
```

Conclusão

Tabela de comparação entre os algoritmos implementados.

Coluna	Significado	Interpretação
Algoritmo	O algoritmo de busca utilizado	GBFS, BFS, IDFS, A*, IDA*
Estados Testados	Número de instâncias do 8-puzzle resolvidas	Todas as linhas mostram 100, ou seja, cada algoritmo foi testado em 100 estados iniciais
Média Nós Expandidos	Número médio de nós gerados/explorados durante a busca	Indica o esforço computacional. Quanto menor, mais eficiente a busca em termos de expansão. Ex.: GBFS expandiu ~392 nós em média, enquanto IDFS expandiu ~2,5 milhões, mostrando que IDFS é muito custoso em termos de nós.
Média Comprimento da Solução	Número médio de movimentos da solução final	Indica a qualidade da solução. BFS, IDFS, A* e IDA* encontram soluções próximas ou iguais ao ótimo (~22), enquanto GBFS tem soluções mais longas (~140), porque é guloso e não garante ótimo.
Média Tempo (s)	Tempo médio de execução por instância	Mede eficiência temporal. GBFS e A* são extremamente rápidos (0.0011 e 0.0028 s), enquanto BFS leva mais tempo (~0.19 s) e IDFS ~0.0659 s.
Média h	Média do valor da função heurística durante a execução	Avalia como a heurística influenciou a busca. GBFS começou com média ~6.9, A* com ~10, IDA* ~13.88. BFS e IDFS não usam heurística para decisão, então h média = 0.
h Inicial (média)	Valor médio da heurística no estado inicial	Todos os algoritmos usam os mesmos estados iniciais, então o valor inicial da heurística é o mesmo (13.88).

Resumo da interpretação

1. GBFS

- Expande poucos nós e é muito rápido, mas produz soluções **bem mais longas que o ótimo**.
- É eficiente em tempo, mas não garante qualidade ótima.

2. BFS

- Garante **soluções ótimas**, mas expande **muitos nós**, sendo mais lento e usando mais memória.
- Não usa heurística.

3. IDFS

- Também garante solução ótima, mas é extremamente custoso em número de nós expandidos e memória da pilha.
- Heurística não usada para decisão.

4. A*

- Combina heurística e custo do caminho ($f = g + h$), produz **soluções ótimas** com muito menos nós expandidos que BFS/IDFS (~895 nós).
- Rápido e eficiente.

5. IDA*

- Variante de A* com profundidade iterativa. Garante solução ótima e mantém baixo uso de memória. Expande mais nós que A*, mas muito menos que IDFS ou BFS.
 - Soluções ótimas e tempo razoável.
-
- Algoritmos informados com heurística (A*, IDA*, GBFS) expandem **menos nós** e são **mais rápidos**, mas apenas A* e IDA* garantem soluções ótimas.
 - BFS e IDFS garantem ótimos, mas com **alto custo computacional**.
 - GBFS é rápido e econômico em nós, mas soluções podem ser muito mais longas.